RTL Track

Lab T-RTL1: Clock Domain Crossing Library

Synchronizers, Asynchronous FIFO and Multi-Clock Verification

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Build a library of reusable clock-domain-crossing blocks in `hw/ip/cdc/`, verify them with multi-clock testbenches that model synchronizer uncertainty, and constrain them correctly for static timing analysis.

Duration	2 weeks (approximately 30–40 hours)
Prerequisites	Common core: Labs R1–R4, V1–V3, P1–P3 (in particular R2 for scheduling, V1/V2 for SVA and regressions, P2 for STA and SDC)
Designs	`cdc_sync`, `reset_sync`, `cdc_pulse`, `cdc_handshake`, `async_fifo` (all new)
Reused in	T-RTL2 (multi-clock integration of the data cache)

Contents

1 Overview **3**

2 CDC Fundamentals **3**

2.1 Metastability . . . 3
2.2 The Two-Flip-Flop Synchronizer . . . 4
2.3 Why Multi-Bit Values Cannot Be Synchronized Bit by Bit . . . 5
2.4 Pulses . . . 5
2.5 Reconvergence . . . 5
2.6 Reset . . . 6
2.7 Choosing a Method . . . 6

3 Library Specification **6**

3.1 Blocks . . . 6
3.2 Interfaces . . . 7
3.3 Asynchronous FIFO . . . 7
3.4 Handshake Synchronizer . . . 8
3.5 RTL Requirements . . . 9

4 Verifying CDC Logic **9**
4.1 Why RTL Simulation Hides CDC Bugs 9
4.2 Synchronizer Delay Model 10
4.3 Multi-Clock Testbenches 10
4.4 Assertions 13
4.5 Structural CDC Check 14

5 Timing Constraints for CDC **15**

6 Requirements and Tasks **15**

7 Repository and Flow **16**

8 Deliverables **17**

9 Assessment **18**

10 Review Questions **18**

11 References **19**

1 Overview

Every block in the common core used a single clock. Real chips do not: the core, the memory interface, the debug port and the peripherals usually run on different clocks, and signals must cross between them. A signal that crosses from one clock domain to another without proper synchronization causes failures that are rare, depend on temperature and voltage, are almost impossible to reproduce — and are *invisible in ordinary RTL simulation*. This makes clock domain crossing (CDC) one of the most important topics in digital design, and one where a single careless engineer can cost a tape-out.

In a real team, CDC blocks are not written ad hoc in every design. They live in a small, heavily reviewed library (OpenTitan's `prim_flop_2sync`, `prim_fifo_async`, `prim_pulse_sync`; PULP `common_cells`' `cdc_2phase`, `cdc_fifo_gray`), and the design rule is simply: *every crossing goes through a library cell.* In this lab you write that library:

- a bit synchronizer, a reset synchronizer, a pulse synchronizer, a handshake synchronizer and an asynchronous FIFO, in `hw/ip/cdc/`;
- a multi-clock verification environment with randomized clocks and a synchronizer delay model, plus assertions per clock domain;
- timing constraints for the asynchronous FIFO and an STA run that shows what they do.

Where this lab sits. It builds on Lab R2 (simulation semantics and races, now with two clocks), Lab V1 and Lab V2 (SVA, plusargs, regressions, scoreboards) and Lab P2 (SDC, OpenSTA). Nothing is re-taught here: you apply those tools to a problem they were not designed for. Lab T-RTL2 uses this library, unchanged, to control the data cache from a second clock domain, and extends the per-block crossing tables of this lab into a full CDC crossing report.

By the end of this lab you should be able to:

1. Explain metastability, why two flip-flops reduce its risk, and what mean time between failures (MTBF) depends on.
2. Choose the correct synchronization method for a level, a pulse, a multi-bit value and a data stream.
3. Explain why a binary counter must not cross clock domains and why a Gray-coded counter can.
4. Design and verify an asynchronous FIFO, including a derivation of its full and empty conditions.
5. Explain why RTL simulation hides CDC bugs, and expose them with randomized clocks and a synchronizer delay model.
6. Write assertions for multi-clock logic, each clocked in the right domain.
7. Constrain asynchronous clocks for static timing analysis and explain what the constraint stops checking.

2 CDC Fundamentals

2.1 Metastability

A flip-flop guarantees a clean output only if its input is stable during the setup and hold window around the clock edge. In a single clock domain, static timing analysis guarantees this (Lab P2). When the input comes from another, unrelated clock, the input will sooner or later change inside that window. The flip-flop may then enter a *metastable* state: its output hovers between 0 and 1 for an unpredictable time before it resolves to either value.

Two facts follow:

- Metastability cannot be prevented, only made unlikely. The probability that a flip-flop is still metastable falls exponentially with the time it is allowed to resolve.
- Even after resolving, the value captured may be the old or the new one. The destination cannot know in which cycle a change arrives — only that it arrives within a bounded number of cycles.

The reliability of a synchronizer is expressed as mean time between failures:

$$MTBF = \frac{e^{t_r/\tau}}{T_W \cdot f_{clk} \cdot f_{data}}$$

where t_r is the time available for resolution, τ and T_W are properties of the flip-flop, f_{clk} is the destination clock frequency and f_{data} is the rate of data changes. Adding a synchronizer stage adds almost a full clock period to t_r, which multiplies MTBF by a huge factor. This is why two stages are standard and three are used at high frequencies.

Worked example (illustrative numbers). Take $\tau = 50\,\text{ps}$, $T_W = 100\,\text{ps}$, $f_{clk} = 100\,\text{MHz}$ and $f_{data} = 10\,\text{MHz}$. If logic after a single capturing flip-flop leaves only $t_r = 1\,\text{ns}$, then $MTBF = e^{20}/(10^{-10} \cdot 10^8 \cdot 10^7) \approx 4.9 \cdot 10^8/10^5 \approx 4.9 \cdot 10^3\,\text{s}$: *about one failure every 80 minutes.* A second flip-flop with nothing between the stages adds roughly 9.5 ns, so the exponent grows by 190 and MTBF exceeds the age of the universe by many orders of magnitude. Recompute both cases at 500 MHz in your `doc/design.md`; the real τ and T_W of a sky130 flip-flop are not in the liberty file, so state your assumptions.

2.2 The Two-Flip-Flop Synchronizer

Figure 1 shows the circuit and what happens when the source changes inside the setup/hold window of a destination edge: the first stage goes metastable and has (almost) one full destination period to resolve before the second stage samples it.

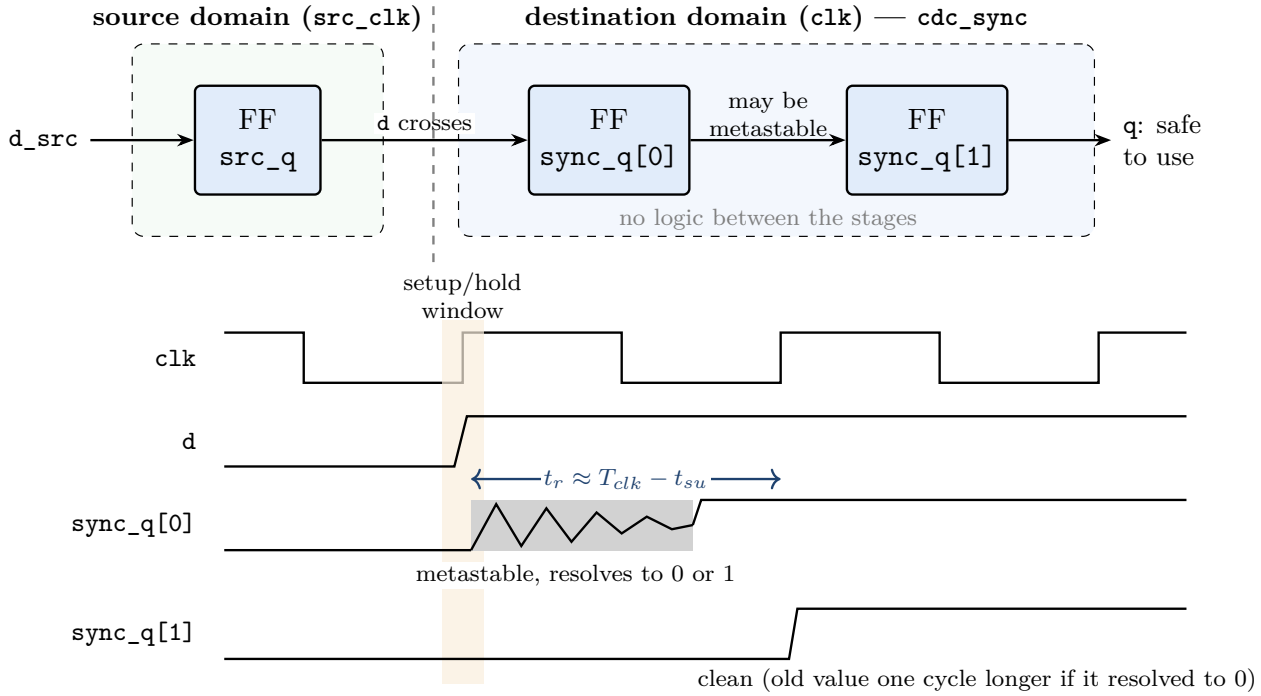


Figure 1: Two-flip-flop synchronizer (top) and a source change inside the setup/hold window of a destination edge (bottom): `sync_q[0]` may be metastable for up to about one destination period; `sync_q[1]` samples it only after that resolution time.

Rules for a correct single-bit synchronizer:

1. The signal must come **directly from a flip-flop** in the source domain. Combinational logic before the synchronizer can glitch, and the destination may capture the glitch.
2. **No logic between the synchronizer stages.** The first stage needs the whole period to resolve.
3. The output of the first stage is used **only** by the second stage.
4. The source value must stay stable long enough to be captured: at least about 1.5 destination clock periods.

2.3 Why Multi-Bit Values Cannot Be Synchronized Bit by Bit

Put a 4-bit binary counter through four parallel synchronizers. When the counter goes from 7 (`0111`) to 8 (`1000`), all four bits change. Each synchronizer may capture its bit's old or new value independently, so the destination may see any of the 16 values for one cycle — for example `1111` = 15. The value was never in the source.

Three safe solutions exist, each for a different situation:

Method	Idea	Use when
Gray code	Consecutive values differ in exactly one bit; an uncertain capture yields either the old or the new value, both valid	The value is a counter that changes by ±1 (FIFO pointers)
Handshake	Hold the data stable; synchronize only a single control bit; the destination samples the data after the control bit arrives	An occasional multi-bit value (a configuration word, a counter snapshot)
Asynchronous FIFO	Store data in memory; synchronize only Gray-coded pointers	A continuous stream of data

2.4 Pulses

A one-cycle pulse from a fast domain can fall entirely between two edges of a slow destination clock and be lost. A *toggle* (pulse) synchronizer converts each source pulse into a change of a level, which is synchronized and turned back into a pulse by an edge detector. Pulses closer together than about two destination cycles still merge, so the source must respect a minimum spacing — a rule you will express as an `assume property`.

Figure 2 shows the structure of `cdc_pulse`.

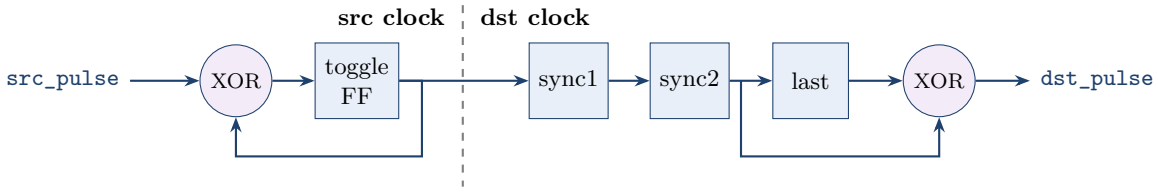


Figure 2: Toggle pulse synchronizer `cdc_pulse`: each source pulse flips a level (toggle flip-flop), the level crosses through `cdc_sync` (`sync1`, `sync2`), and an XOR of the synchronized level with its delayed copy (`last`) restores one pulse.

2.5 Reconvergence

Two signals synchronized separately may arrive in different destination cycles, even if they changed in the same source cycle. Logic that combines them after synchronization can see an impossible

combination. Synchronize related signals together through one mechanism (a handshake or a FIFO), never through separate synchronizers.

2.6 Reset

An asynchronous reset is safe to *assert* at any time, but its *release* is a signal crossing into each clock domain: if it is released near a clock edge, some flip-flops leave reset one cycle later than others (a recovery/removal violation, the check you met in Lab P2). Every clock domain therefore has a reset synchronizer: asynchronous assertion, synchronous deassertion.

```
// reset_sync: the chain's D input is tied high; the raw reset clears it.
always_ff @(posedge clk or negedge arst_n) begin
  if (!arst_n) sync_q <= '0;
  else         sync_q <= {sync_q[STAGES-2:0], 1'b1};
end
assign sync_rst_n = sync_q[STAGES-1];
```

2.7 Choosing a Method

What crosses	Library cell	Condition that must hold
Slow-changing single-bit level (enable, mode)	`cdc_sync`	Held for $\geq$ 1.5 destination periods; no other bit depends on it arriving in the same cycle
Single event (start, done, interrupt)	`cdc_pulse`	Source pulses spaced by the minimum distance
Occasional multi-bit word	`cdc_handshake`	Source waits for `src_ready`; data stable during the transfer
Stream of words	`async_fifo`	Producer respects `wfull`, consumer respects `rempty`
Counter that changes by ± 1	`cdc_sync` on a Gray value	Gray value registered in the source domain; bounded skew (Section 5)
Reset release	`reset_sync`	One per clock domain

3 Library Specification

3.1 Blocks

Module	Function
`cdc_sync #(WIDTH, STAGES)`	Bit synchronizer; each bit independent. Only for single bits, Gray-coded values or bits that are allowed to arrive in different cycles
`reset_sync #(STAGES)`	Asynchronous assertion, synchronous deassertion of an active-low reset
`cdc_pulse`	Toggle-based pulse synchronizer: one source pulse produces one destination pulse
`cdc_handshake #(WIDTH)`	Four-phase request/acknowledge transfer of a `WIDTH`-bit value; `src_valid`/`src_ready` on the source side, `dst_valid` pulse on the destination side
`async_fifo #(WIDTH, DEPTH)`	Asynchronous FIFO, `DEPTH` a power of two, Gray-coded pointers, `wfull` in the write domain and `rempty` in the read domain

3.2 Interfaces

Port names carry the domain as a prefix (`src_`/`dst_`, `w`/`r`), so that a reviewer can see from every connection in which domain a signal lives. Defaults: `STAGES = 2`, `WIDTH = 1` for `cdc_sync`, `WIDTH = 32` for the others, `DEPTH = 8`.

Module	Domain	Ports
`cdc_sync`	dst	`clk`, `rst_n`, `d[WIDTH]` (from a source flop), `q[WIDTH]`
`reset_sync`	dst	`clk`, `arst_n` (raw asynchronous reset), `sync_rst_n`
`cdc_pulse`	src	`src_clk`, `src_rst_n`, `src_pulse`
	dst	`dst_clk`, `dst_rst_n`, `dst_pulse`
`cdc_handshake`	src	`src_clk`, `src_rst_n`, `src_valid`, `src_ready`, `src_data[WIDTH]`
	dst	`dst_clk`, `dst_rst_n`, `dst_valid` (one-cycle pulse), `dst_data[WIDTH]` (held until the next transfer)
`async_fifo`	write	`wclk`, `wrst_n`, `w_valid`, `w_ready` (= `!wfull`), `w_data[WIDTH]`, `wfull`
	read	`rclk`, `rrst_n`, `r_valid` (= `!rempty`), `r_ready`, `r_data[WIDTH]`, `rempty`

The FIFO uses the valid/ready convention of the program on both sides: a write is accepted on a `wclk` edge with `w_valid && w_ready`, a read on an `rclk` edge with `r_valid && r_ready` (first-word fall-through, as `sync_fifo` in Lab R2). Both resets come from one asynchronous source, synchronized into each domain by `reset_sync` outside the FIFO; explain in `doc/design.md` what happens if only one side is reset.

3.3 Asynchronous FIFO

Figure 3 is the block diagram to design: two clock domains, the storage between them, a binary and a Gray pointer per side, the two pointer synchronizers and the full/empty logic.

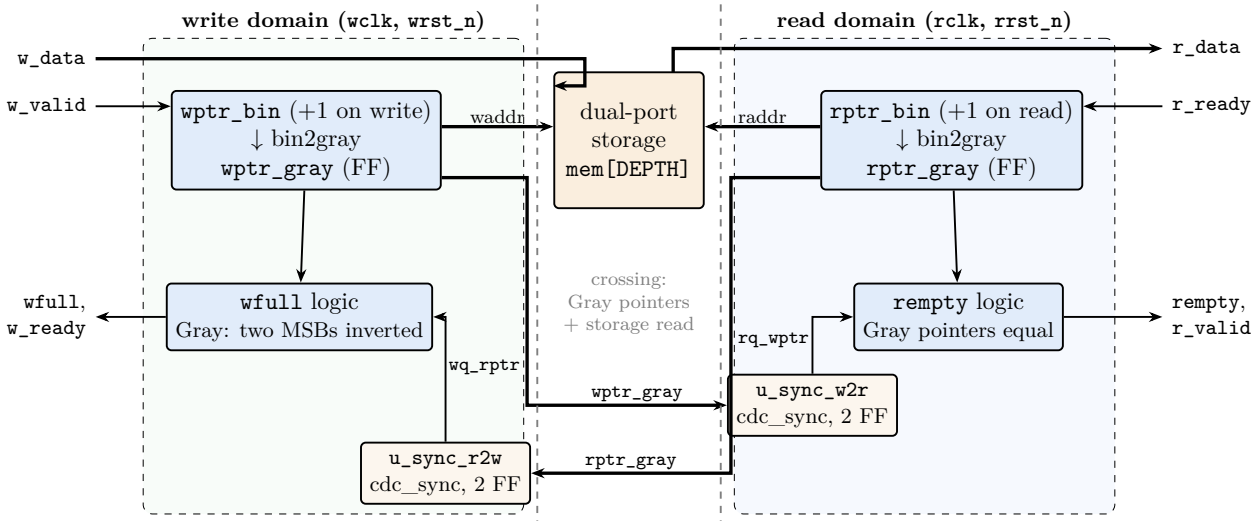


Figure 3: Asynchronous FIFO `async_fifo`: the storage is written in `wclk` and read (first-word fall-through) in `rclk`; only the registered Gray pointers cross, each through a `cdc_sync` in the receiving domain; `wfull` and `rempty` are computed locally from the own pointer and the synchronized (older) other pointer.

Key points:

- Pointers have one bit more than the address (`$clog2(DEPTH)+1`). The extra bit distinguishes "full" from "empty" when the address bits are equal.

- Only Gray-coded pointers cross the domains, each through a `cdc_sync` in the receiving domain. The Gray pointer is registered in the sending domain (rule 1 of the synchronizer).
- `wfull` is computed in the write domain from the local write pointer and the synchronized read pointer; `rempty` in the read domain from the local read pointer and the synchronized write pointer. Both are *pessimistic*: because the other pointer is a few cycles old, the FIFO may report full or empty slightly too long, but never too short. Explain why this is safe.
- Derive the Gray-code full condition yourself and explain it in `doc/design.md`. It is not simply "pointers equal except the most significant bit".
- The memory is written in the write domain and read combinationally (or registered in the read domain) at the read address. The data path `mem` → `r_data` is itself a crossing; explain why it needs no synchronizer.
- `DEPTH` must be a power of two; add an elaboration-time check (`if (DEPTH & (DEPTH-1)) $error(...)` in a `generate` block). Explain what breaks for other depths (compare with the non-power-of-two `sync_fifo` of Lab R2).

```
function automatic logic [N-1:0] bin2gray(logic [N-1:0] b);
  return b ^ (b >> 1);
endfunction

function automatic logic [N-1:0] gray2bin(logic [N-1:0] g);
  logic [N-1:0] b;
  b[N-1] = g[N-1];
  for (int i = N-2; i >= 0; i--) b[i] = b[i+1] ^ g[i];
  return b;
endfunction
```

3.4 Handshake Synchronizer

The four-phase protocol (Figure 4), with `req` synchronized into the destination and `ack` synchronized back:

1. The source registers the data and raises `req`. The data register must not change until the transfer completes.
2. The destination sees the synchronized `req` rise, captures the data (now stable for at least two destination cycles), pulses `dst_valid` and raises `ack`.
3. The source sees the synchronized `ack` and lowers `req`.
4. The destination sees `req` fall and lowers `ack`. The source may start the next transfer when it sees `ack` low.

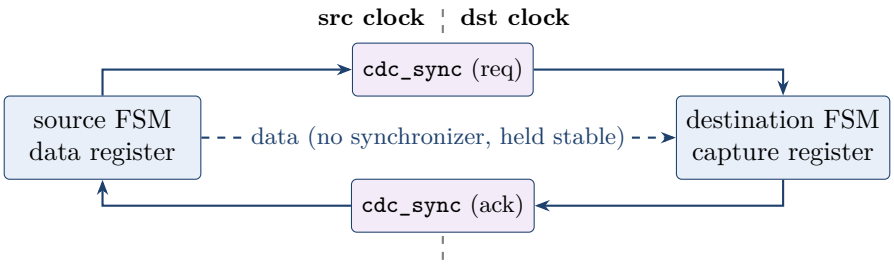


Figure 4: Four-phase handshake synchronizer `cdc_handshake`: only `req` and `ack` cross, each through a `cdc_sync`; the data register is held stable and captured without a synchronizer (Figure 1 for the synchronizer itself).

Measure and report the transfer latency in source and destination cycles for at least three clock ratios. Explain why a handshake is unsuitable for a continuous data stream. A two-phase (toggle) handshake is a valid alternative; if you choose it, explain the trade-off in the README.

3.5 RTL Requirements

- All coding rules of Lab R1 apply; the library is lint-clean with `-Wall`, and every waiver in `lint/cdc.vlt` has a justification.
- Every synchronizer flip-flop lives inside `cdc_sync` or `reset_sync`; other blocks instantiate them and never write their own synchronizer chains. The register is named `sync_q` so that constraints and the crossing review can find it by name (`*u_sync*/sync_q*`).
- Every signal entering a `cdc_sync` is driven directly by a flip-flop of the source domain.
- `STAGES` ≥ 2 is checked at elaboration time.
- Assertions are inside `` `ifndef SYNTHESIS ``; the delay model (Section 4.2) is never synthesized.

On the Synopsys cloud: Lint with SpyGlass: `make lint IP=cdc LINT_TOOL=spyglass` (goal `lint/lint_rtl`; the cloud default) or VC SpyGlass (`LINT_TOOL=vcst`); Verilator stays available with `LINT_TOOL=verilator`. Lint all five blocks in one run through a small wrapper that instantiates each of them between two clock domains (`LINT_TOP`, `LINT_FLIST`); the same wrapper is the top of the structural CDC check (Section 4.5).

4 Verifying CDC Logic

4.1 Why RTL Simulation Hides CDC Bugs

A simulator never makes a flip-flop metastable and never lets bits of a bus arrive in different cycles: every flip-flop captures the new value exactly when the clock edge comes after the data change. A design with a binary pointer instead of a Gray pointer, or with no synchronizer at all, will usually pass RTL simulation. CDC verification therefore needs three things that functional simulation alone does not provide:

1. **Structural review:** every crossing is listed and uses an approved synchronizer. Commercial flows use dedicated CDC analysis tools for this; here you do it by hand, in the crossing table of each block (Section 8), and, where SpyGlass CDC is available, with `make cdc` (Section 4.5). Lab T-RTL2 extends it to a full CDC crossing report.
2. **A synchronizer delay model** that makes simulation as uncertain as the hardware.
3. **Randomized clocks**: frequency ratios and phases that change from seed to seed.

4.2 Synchronizer Delay Model

When a source signal changes shortly before a destination edge, the first synchronizer stage may capture either the old or the new value. Model this in `cdc_sync`, for simulation only, by choosing per bit at random between the current input and the input of the previous edge:

```
`ifdef CDC_RANDOM_DELAY
  // Simulation only: each bit independently may arrive one cycle late,
  // as if the first stage went metastable and resolved to the old value.
  logic [WIDTH-1:0] d_q;
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      d_q       <= '0;
      sync_q[0] <= '0;
    end else begin
      d_q <= d;
      for (int i = 0; i < WIDTH; i++)
        sync_q[0][i] <= ($urandom_range(1) == 1) ? d[i] : d_q[i];
    end
  end
`else
  // normal first stage: sync_q[0] <= d
`endif
```

The synthesis flow must never see the model: make the combination of `SYNTHESIS` and `CDC_RANDOM_DELAY` an elaboration error.

With this model, a multi-bit binary value really does arrive as an impossible mixture, exactly as in hardware. A design that depends on an exact synchronizer latency, or on several bits arriving together, now fails in simulation. The model is enabled with `DEFINES=CDC_RANDOM_DELAY`; the regression must run with it. The random choice uses the simulator's seed, so a failure is reproducible with the same `SEED` (Lab V1).

Test term — Regression. The whole list of tests × seeds × configurations run by one command (`make regress IP=cdc`) with a pass/fail summary, as in Lab V1. Here a configuration is a testbench (`TB_TOP`) with or without the delay model, and the clock ratio is a second axis: every block, every ratio class, at least five seeds. A pass proves the blocks across clock ratios and synchronizer outcomes, not that the design has no structural CDC bug.

What the model can and cannot do. The model mixes the current input with the input of the *previous destination edge*. If the source changes several times between two destination edges, that mixture can be a value that real hardware (where only the bits switching at the sampling instant are uncertain) cannot produce; drive multi-bit test values at most once per destination period. Conversely, the model does not make every CDC bug visible: a FIFO whose pointers cross in *binary* usually still passes the scoreboard with the model on, because its equality-based flags and one-entry-per-cycle pointers tolerate the mixtures. The Gray assertion and the structural check (Section 4.5) catch it — run all three.

4.3 Multi-Clock Testbenches

Write the clock generation once, as a small module `dv/common/clk_rst_gen.sv` (shared, design-independent; Lab T-RTL2 reuses it), and instantiate one per domain. The core of it:

```
`timescale 1ns/1ps
module tb_async_fifo;
  logic    wclk = 0, rclk = 0;
  realtime wclk_half, rclk_half, rphase;
```

```
  initial begin
    real wp, rp, ph;
    // default: random periods between 5 and 15 ns, random phase
    wp = $urandom_range(5000, 15000) / 1000.0;
    rp = $urandom_range(5000, 15000) / 1000.0;
    ph = $urandom_range(0, 5000)     / 1000.0;
    void'($value$plusargs("WCLK_NS=%f",   wp));
    void'($value$plusargs("RCLK_NS=%f",   rp));
    void'($value$plusargs("RPHASE_NS=%f", ph));
    $display("CLOCKS: WCLK=%0.3f ns RCLK=%0.3f ns RPHASE=%0.3f ns", wp, rp, ph);
    wclk_half = wp / 2;
    rclk_half = rp / 2;
    rphase    = ph;
  end

  initial begin #0.001; forever #(wclk_half) wclk = ~wclk; end
  initial begin #0.001; #(rphase); forever #(rclk_half) rclk = ~rclk; end
  // ...
endmodule
```

The clock periods and phase are part of the configuration that every simulation prints at the start (program convention), next to `TEST`, `SEED` and the plusargs.

Test term — Random test. Here the randomness is in the *clocks* as well as in the data: each seed draws other periods, phases, idle cycles and synchronizer outcomes (delay model). A failure is reproduced with the same `SEED`; a pass over many seeds covers edge alignments nobody would write by hand.

Requirements:

- Each domain has its own reset synchronizer, driver and monitor. Drivers use `<=` at the rising edge of *their own* clock (the race-free driving standard of Lab R2 applies per domain).
- The FIFO scoreboard is a queue: the write-side monitor pushes every accepted write, the read-side monitor pops and compares every accepted read. Check that no item is lost, duplicated or reordered.
- The handshake scoreboard compares every `dst_valid` value with the oldest outstanding source value; the pulse scoreboard counts source and destination pulses.
- The regression must cover: write clock much faster than read clock, much slower, nearly equal (for example 10.0 and 10.1 ns), and equal frequency with a phase offset. Every configuration runs with `CDC_RANDOM_DELAY` and at least 5 seeds.
- The FIFO tests must fill it completely, empty it completely and run with both sides active at full rate for long periods. Random idle cycles on both sides (`+W_IDLE_PCT=`, `+R_IDLE_PCT=`) cover the cases in between (boxes below).
- The end of test waits until the FIFO has drained and the scoreboard queue is empty; a watchdog in *time*, not in cycles of one clock, catches hangs.

Test term — Corner-case test. Full and empty are the corners of a FIFO: fill it with the reader stopped until `wfull` (exactly `DEPTH` words must be accepted), then drain it with the writer stopped until `rempty`. Most pointer and flag bugs show up only here.

Test term — Stress / back-pressure test. Stress: both sides at full rate for thousands of words, so that the pointers change on every edge and the flags toggle constantly (this test also measures throughput). Back-pressure: random idle cycles on the write side and random `r_ready` on the read side exercise every stall and every full/empty transition in between.

Test term — Smoke test. The first, shortest test of every testbench: a few transfers at one clock ratio and the flags after reset. It proves that the block is alive and the two-clock environment works; nothing more.

Figure 5 shows the structure every testbench of the library follows, and Table 1 lists the tests of the reference environment.

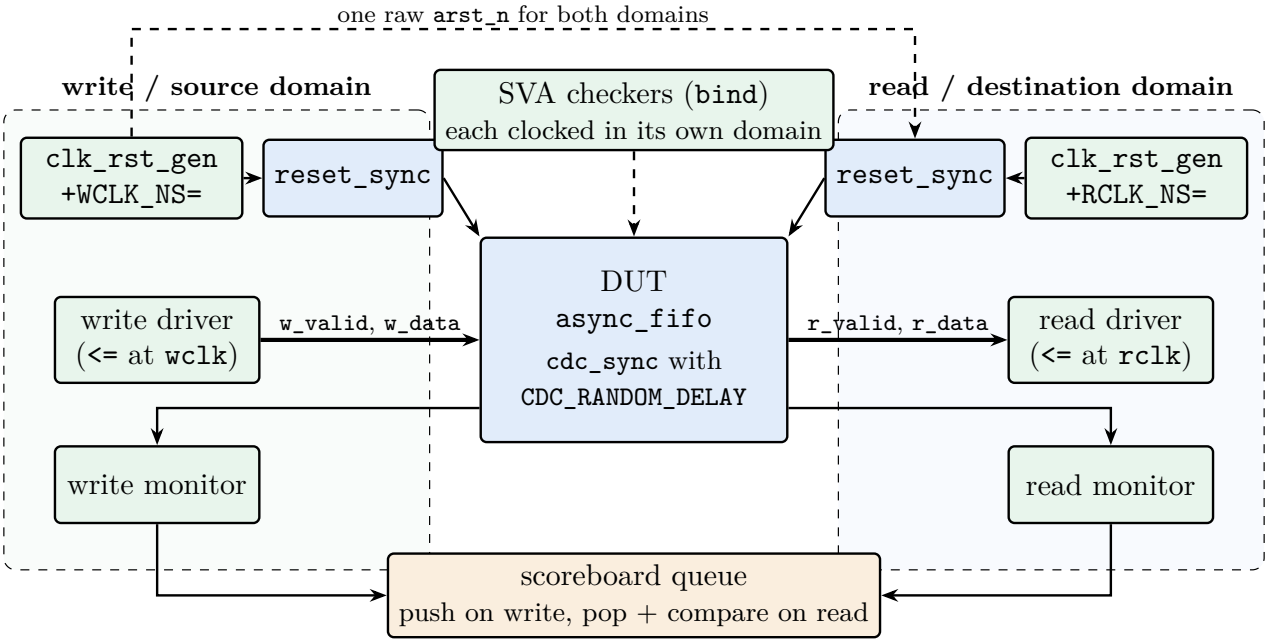


Figure 5: Multi-clock testbench of the CDC library (here `tb_async_fifo`): one clock generator, reset synchronizer, driver and monitor per domain, one raw asynchronous reset, a queue scoreboard fed by both monitors, assertion checkers bound to the DUT, and the delay model inside every `cdc_sync`.

Testbench	Test	What it shows
`tb_async_fifo`	`smoke`	a few words written, read back; flags after reset (the smoke test: is it alive?)
	`fill_drain`	corner case: exactly `DEPTH` words until `wfull`, exactly as many read until `rempty`, three times
	`stream`	stress: both sides at full rate; throughput per cycle of the slower clock
	`random`	back-pressure: random idle cycles on both sides
`tb_cdc_sync`	`smoke`	a level arrives, in order, within the latency bound
	`gray`	a Gray counter never arrives as an impossible value
	`binary`	negative: a binary counter *does* (with the delay model)
`tb_cdc_pulse`	`smoke`, `burst`	one destination pulse per source pulse; `burst` at the minimum spacing
	`merge`	negative: pulses too close — the spacing assumption must fire
`tb_cdc_handshake`	`random`, `back_to_back`	data integrity; a new transfer in the first cycle `src_ready` returns
`tb_reset_sync`	`smoke`	random assertion/release times: asynchronous assertion, release at an edge
Clock-ratio class as a suffix of the test name (one log per run in a regression): `_fast2slow` (3/17 ns), `_slow2fast` (17/3 ns), `_near` (10.0/10.1 ns), `_phase` (10/10 ns with a phase offset); no suffix = random clocks.		

Table 1: Tests of the reference environment (names used in `dv/regress.list`).

On the Synopsys cloud: Run the multi-clock testbenches with VCS: `make sim IP=cdc SIM=vcs` (the cloud default). VCS is four-state (an unreset synchronizer shows X), schedules `always @(posedge)` monitors and concurrent assertions with the IEEE preponed values in every domain, and supports the `realtime` clock arithmetic, queues and `bind` used here. Verilator 5.013 (installed) compiles the library and lints it clean, but its monitors and concurrent assertions see post-edge values, so the multi-clock environment reports false errors; use VCS until Verilator $\geq$ 5.020 is installed. Waves: `make sim WAVES=1` then `make waves` (Verdi, both clock domains in one FSDB).

4.4 Assertions

Test term — Assertion (SVA) checks. Properties checked at every edge of their own clock while any test runs. Here they catch what the scoreboard cannot see: a pointer that changes more than one bit, data that moves while it is being handshaked, a reset released off a clock edge. Cover properties record that the interesting cases (full, empty, back-to-back) happened at all.

Each assertion is clocked by the clock of the domain whose signals it checks. A property that samples signals of the other domain is itself an unsynchronized crossing and will give misleading results. The checkers live in `hw/ip/cdc/dv/sva/` and are attached with `bind`, as in Lab V1.

1. **Gray property:** in the write domain, `$countones(wptr_gray ^ $past(wptr_gray)) <= 1`; the same for the read pointer in the read domain.
2. **FIFO:** no write accepted while `wfull`; no read accepted while `rempty`.
3. **Handshake:** the source data is stable while `req` is high; `req` rises only while the synchronized `ack` is low.

4. **Pulse synchronizer:** an `assume property` for the minimum spacing of source pulses; an assertion that every source pulse produces exactly one destination pulse (count them).
5. **Reset synchronizer:** the synchronized reset is released only at a rising edge of its clock.

Cover properties: FIFO full in the write domain; FIFO empty in the read domain; a write and a read in the same simulation time step; a handshake transfer started immediately after the previous one completed. The support of the installed Verilator for concurrent assertions and cover properties is the one established in Lab V1; use the same fallback where needed.

4.5 Structural CDC Check

Simulation, even with the delay model, only shows the crossings the tests happen to exercise. A *structural* CDC tool reads the RTL, finds every flip-flop whose data comes from another clock domain and classifies each crossing: synchronized by a recognized scheme (multi-flop synchronizer, handshake/enable, FIFO), or unsynchronized. It also checks the synchronizer rules of Figure 1: logic between synchronizer stages, combinational logic before a synchronizer, convergence of separately synchronized signals (reconvergence), and reset synchronization. It needs constraints the RTL does not contain: which ports are clocks, which clocks are in the same domain, which signals are resets, and in which domain every primary input lives.

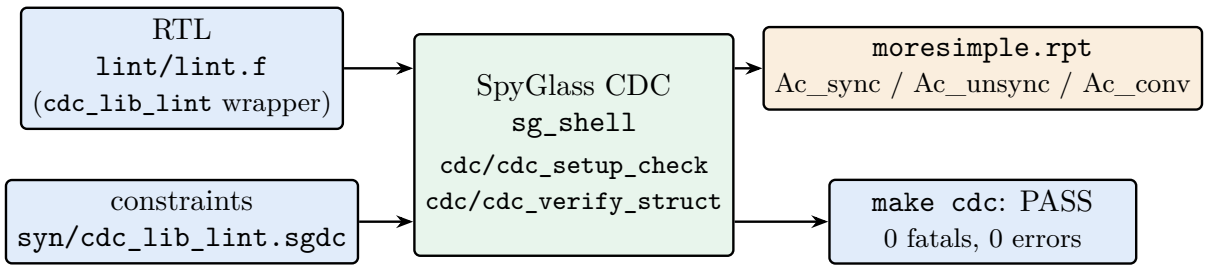


Figure 6: Structural CDC check: the RTL and the SpyGlass design constraints (SGDC) go into SpyGlass CDC; the target fails on any fatal or error.

There is no open-source equivalent of a structural CDC tool; without one, the crossing table (Section 8) *is* the structural check, done by hand and reviewed.

On the Synopsys cloud: SpyGlass CDC is licensed on the cloud: `make cdc IP=cdc` (new target, `flow/mk/cdc.mk`; script `flow/cdc/spyglass_cdc.prj`; output in `build/cdc/<cfg>/cdc/`). It checks the wrapper `lint/cdc_lib_lint.sv`, which instantiates all five blocks between two clock domains, with the constraints `syn/cdc_lib_lint.sgdc`: `clock -name clk_a -domain dom_a`, `reset -name arst_n -value 0`, `abstract_port -ports <input> -clock <clk>` for every input, and `fifo -memory` for the FIFO storage (without it SpyGlass reports `r_data` in the fan-out of two domains, `Clock_sync06`). Expected result on a correct library: 0 errors, every crossing reported as synchronized (`Ac_sync01/02`); read and justify every warning (for example `Ac_conv02`: the bits of a Gray pointer converge on the flag logic — correct because they are Gray-coded). Prove the check works: a deliberately broken top in which a domain-b flip-flop samples a domain-a signal directly must fail with `Ac_unsync01` (`make cdc CDC_TOP=... CDC_EXPECT=fail` passes only if an error is found). Variables: `CDC_TOP`, `CDC_FLIST`, `CDC_SGDC`, `CDC_GOALS` (add `cdc/cdc_verify` for the formal checks), `CDC_FAIL_ON`. The alternative tool is VC SpyGlass CDC in VC Static (`vc_static_shell`).

5 Timing Constraints for CDC

Static timing analysis assumes that every clock is related to every other clock unless told otherwise. For two unrelated clocks, it will compute a nonsensical "worst case" alignment of their edges and report violations on every crossing path.

```
# hw/ip/cdc/syn/async_fifo.sdc
create_clock -name wclk -period 7.0  [get_ports wclk]
create_clock -name rclk -period 10.0 [get_ports rclk]
set_clock_groups -asynchronous -group {wclk} -group {rclk}
# I/O delays and loads as in Lab P2, each port relative to its own clock
```

The generic `flow/sta/sta.tcl` of Lab P2 takes one `CLK_PERIOD`; a two-clock block defines both periods in its SDC and ignores `CLK_PERIOD`. Keep a second file `hw/ip/cdc/syn/async_fifo_noasync.sdc` without the `set_clock_groups` line for task 1.

Tasks:

1. Run STA on `async_fifo` without the `set_clock_groups` line. Which paths fail, and why are these results meaningless? List the startpoints and endpoints of the inter-clock paths (`report_checks -from [get_clocks wclk] -to [get_clocks rclk]`): every endpoint must be a first synchronizer stage or the read data path. This is your first tool-assisted crossing check.
2. Add the line and run again. Confirm that only paths inside each domain remain, and report WNS per clock.
3. Declaring the clocks asynchronous removes *all* checks on the crossing paths. In production flows, Gray-pointer crossings additionally get a maximum-delay constraint so that the bits of a pointer cannot drift more than one destination period apart. Explain in `doc/design.md` why Gray coding is only safe if that skew is bounded.
4. Replace the clock group by `set_max_delay` on the `wptr_gray`→`sync_q` and `rptr_gray`→`sync_q` paths plus `set_false_path` on the remaining crossings. Show in the report which paths are now checked and against which limit. Which constraint wins if both a clock group and a max delay apply to the same path?
5. Synthesize `reset_sync` and `cdc_sync` and find their flip-flops in the netlist. Confirm that synthesis neither merged nor removed any synchronizer stage.

On the Synopsys cloud: Synthesize with Fusion Compiler (`make synth IP=cdc DUT_TOP=async_fifo SYN_TOOL=fc`) and time with PrimeTime (`make sta IP=cdc DUT_TOP=async_fifo STA_TOOL=pt`); both read the same SDC files. PrimeTime's equivalent of OpenSTA's `report_checks -from [get_clocks wclk] -to [get_clocks rclk]` is `report_timing -from [get_clocks wclk] -to [get_clocks rclk]` (or `get_timing_paths` to list the endpoints). In this flow the SDC is also a synthesis input, so `make sta SDC=...` re-synthesizes: compare the netlists too (without the clock group Fusion Compiler upsizes the design to chase the meaningless inter-clock paths). Two PrimeTime precedence facts answer task 4: a false path (and an asynchronous clock group) has priority over a max delay, so never put a clock-to-clock `set_false_path` next to the pointer max delays; and `set_max_delay -datapath_only` excludes clock skew from the bound.

6 Requirements and Tasks

1. Write `doc/spec.md` for the library: one section per block with parameters, ports and protocol (from this document, in your own words).

2. Implement the five blocks in `hw/ip/cdc/rtl/`, lint-clean, following the RTL requirements above. `async_fifo` and `cdc_handshake` instantiate `cdc_sync`; nothing else contains a synchronizer chain.
3. Add the delay model to `cdc_sync`, enabled by `CDC_RANDOM_DELAY`.
4. Write `dv/common/clk_rst_gen.sv` and one testbench per block in `hw/ip/cdc/dv/tb/`, with the drivers, monitors and scoreboards described above, each ending in a single `TEST PASSED`/`TEST FAILED` line.
5. Write the assertion checkers and bind files and the cover properties; show each assertion firing at least once on a deliberately broken copy (as in Lab V1) and record how (error-injection test, box below).
6. Write `dv/testplan.md` mapping each feature (full, empty, pessimism, ordering, pulse merging, handshake back-to-back, reset release) to tests, assertions and cover items.
7. Create `dv/regress.list` covering the four clock-ratio classes for every block, with the delay model and at least 5 seeds each. `make regress IP=cdc` passes.
8. Measure the handshake latency (source and destination cycles) for three clock ratios, and the FIFO throughput at full rate for the four ratio classes.
9. Run the STA tasks of Section 5 and record the results in `doc/ppa.md`.
10. Write the Gray full-condition derivation, the pessimism argument, the MTBF calculation and the crossing table of every block in `doc/design.md`.
11. `make check-all` still passes: no earlier lab is broken.

Test term — Negative / error-injection test. A deliberately broken copy of a block (binary instead of Gray pointers, a data register that is not held, a reset synchronizer without asynchronous clear, a missing synchronizer) is run to prove that the checkers fire. Some CDC bugs are invisible to every simulation check — a missing synchronizer passes the scoreboard — and only the structural check finds them: record which check caught each bug.

7 Repository and Flow

The library is one IP directory containing several modules. The default DUT and testbench are the FIFO; the others are selected on the command line.

```
hw/ip/cdc/
+-- README.md
+-- Makefile
+-- doc/        spec.md, design.md, verification.md, ppa.md
+-- rtl/        cdc_sync.sv, reset_sync.sv, cdc_pulse.sv,
|               cdc_handshake.sv, async_fifo.sv, cdc.f
+-- lint/       cdc.vlt, cdc_lib_lint.sv (wrapper of all blocks), lint.f
+-- dv/
|   +-- testplan.md
|   +-- tb/     tb_cdc_sync.sv, tb_reset_sync.sv, tb_cdc_pulse.sv,
|   |           tb_cdc_handshake.sv, tb_async_fifo.sv
|   +-- env/    per-domain drivers, monitors and scoreboards (tasks)
|   +-- sva/    async_fifo_sva.sv, cdc_handshake_sva.sv, cdc_pulse_sva.sv,
|   |           reset_sync_sva.sv, cdc_bind.sv
|   +-- negative/  broken crossing for 'make cdc' (Synopsys cloud)
|   +-- regress.list
|   +-- dv.f
+-- syn/        async_fifo.sdc, async_fifo_noasync.sdc, async_fifo_maxdelay.sdc,
                cdc_lib_lint.sgdc (SpyGlass CDC constraints, Synopsys cloud)
```

```
dv/common/clk_rst_gen.sv          shared clock/reset generator (new)
```

```
# hw/ip/cdc/Makefile
IP         := cdc
DUT_TOP    ?= async_fifo
TB_TOP     ?= tb_async_fifo
RTL_FLIST  := rtl/cdc.f
TB_FLIST   ?= dv/dv.f
SDC        ?= syn/async_fifo.sdc
PARAMS     ?= WIDTH=8 DEPTH=16
# one model/log directory per TB_TOP
SIM_PER_TB := 1
# lint (and make cdc) all blocks through a wrapper
LINT_TOP   ?= cdc_lib_lint
LINT_FLIST ?= lint/lint.f
include $(REPO_ROOT)/flow/mk/common.mk
include $(REPO_ROOT)/flow/mk/lint.mk
include $(REPO_ROOT)/flow/mk/sim_verilator.mk
include $(REPO_ROOT)/flow/mk/regress.mk
include $(REPO_ROOT)/flow/mk/syn.mk
include $(REPO_ROOT)/flow/mk/sta.mk
# Synopsys cloud: make cdc (SpyGlass CDC)
include $(REPO_ROOT)/flow/mk/cdc.mk
```

(Write comments on their own lines in a real Makefile: make keeps the spaces before a `#` in the value of a variable.)

No new make targets are needed for the open-source flow (the Synopsys cloud adds `make cdc`, Section 4.5), but the flow must be extended *generically* (in `flow/mk/`, not in the IP Makefile) where it assumed one testbench per IP:

- The simulation build directory must depend on `TB_TOP`, otherwise two testbenches overwrite each other's model. For example: `build/cdc/<cfg>/sim/<TB_TOP>/`, switched on by a block variable (`SIM_PER_TB := 1`) so that the one-testbench blocks keep their paths.
- `dv/regress.list` entries must be able to set `TB_TOP` and `PLUSARGS` per line, in addition to the test, seeds and `PARAMS` of Lab V1.

Example commands:

```
make lint  IP=cdc
make sim   IP=cdc TB_TOP=tb_async_fifo DEFINES=CDC_RANDOM_DELAY \
           PLUSARGS="+WCLK_NS=3.0 +RCLK_NS=17.0" SEED=7
make sim   IP=cdc TB_TOP=tb_cdc_handshake PARAMS="WIDTH=32" \
           DEFINES=CDC_RANDOM_DELAY TEST=back_to_back
make regress IP=cdc
make synth IP=cdc DUT_TOP=async_fifo PARAMS="WIDTH=8 DEPTH=16"
make sta   IP=cdc DUT_TOP=async_fifo SDC=syn/async_fifo_noasync.sdc
make sta   IP=cdc DUT_TOP=async_fifo
make cdc   IP=cdc                          # Synopsys cloud: SpyGlass CDC
```

Branch `lab/trtl1-cdc`, tag `trtl1-done` after the review.

8 Deliverables

The merge request contains everything in `hw/ip/cdc/`, the shared `dv/common/clk_rst_gen.sv` and the generic `flow/mk/` changes. Every number in the documents must be reproducible with `make`

from a clean checkout.

- `README.md`: what each block is for and when *not* to use it, how to run every target, results summary, known limitations.
- `doc/spec.md`: parameters, ports and protocol of each block.
- `doc/design.md`: block diagrams; the Gray full-condition derivation; why full/empty are pessimistic and why that is safe; the MTBF calculation; the **crossing table** of every block, in this format (one row per crossing signal):

Signal	From	To	Synchronizer	Why it is safe
`wptr_gray`	wclk	rclk	`cdc_sync`, 2 stages	Registered in wclk; Gray, one bit changes per write; skew bounded by max delay
`mem[i]` → `r_data`	wclk	rclk	none (data)	Entry written before `wptr_gray` advances; read only when the synchronized pointer shows it
...				

- `doc/verification.md`: testplan status, regression summary (all clock-ratio classes, with the delay model, 5 seeds each), assertion and cover results, the "assertion fires" evidence, handshake latency and FIFO throughput measurements.
- `doc/ppa.md`: STA results with and without the clock-group constraint, the inter-clock path list, the max-delay experiment, area of each block.

9 Assessment

Criterion	Weight
CDC library: correctness and structure of each block	30%
Multi-clock verification: delay model, random clocks, scoreboards, regression	20%
Assertions and cover properties	10%
Design documentation: Gray derivation, pessimism, MTBF, crossing tables	15%
Timing constraints and STA	15%
Flow integration, reproducibility and README	10%
Total	100%

10 Review Questions

1. What is metastability, and why can no circuit eliminate it?
2. Why must the input of a synchronizer come directly from a flip-flop?
3. What does a third synchronizer stage cost, and what does it buy? Use your MTBF numbers.
4. Why can a Gray-coded pointer cross with simple synchronizers but a binary pointer cannot? What assumption about skew does this rely on?
5. Why are `wfull` and `rempty` pessimistic, and why is that acceptable? What does the pessimism cost in throughput?

6. When would you use a handshake instead of an asynchronous FIFO?
7. Two synchronized signals that changed together arrive one cycle apart. What is this problem called, and how do you avoid it?
8. Why does the reset need a synchronizer if it is asynchronous?
9. What does `set_clock_groups -asynchronous` tell the timing tool, and what does it stop checking?
10. Your FIFO passed the regression without `CDC_RANDOM_DELAY`. What does that prove, and what does it not prove?

11 References

- Clifford E. Cummings, Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog, SNUG 2008.
- Clifford E. Cummings, Simulation and Synthesis Techniques for Asynchronous FIFO Design, SNUG 2002.
- Harris and Harris, *Digital Design and Computer Architecture: RISC-V Edition*: sections on synchronizers and metastability.
- Onur Mutlu, Digital Design and Computer Architecture lectures: sequential logic timing.
- OpenTitan `hw/ip/prim/rtl/`: `prim_flop_2sync`, `prim_pulse_sync`, `prim_fifo_async`; PULP `common_cells`: `cdc_2phase`, `cdc_fifo_gray`, `sync` — industrial CDC libraries to compare with yours.
- OpenSTA documentation: `set_clock_groups`, `set_max_delay`, `set_false_path` and SDC support.